OPERATING SYSTEMS – HANDWRITTEN EXAM NOTES

UNIT 2: Process, Threads & Concurrency, CPU Scheduling Algorithms, IPC, Gantt Charts & Calculations

B.Tech Mid-1 Examination Preparation | Complete Question Bank Solutions

SECTION A — 2 MARKS QUESTIONS

Question 1

Define Process

Definition & Components

A **Process** is a program in active execution. Unlike a passive program file stored on disk, a process is an active entity executing in memory, comprising:

- **Text Section:** The compiled executable machine code instructions.
- **Current Activity:** Program Counter (PC) and CPU hardware registers.
- **Stack:** Temporary data such as function parameters, local variables, and return addresses.
- **Data Section:** Global and static variables.
- **Heap:** Dynamically allocated memory during runtime.

Question 2

What is meant by the State of a Process?

The **State of a Process** indicates its current execution condition. A process passes through five standard states:

1. **New:** The process is being created and initialized.
2. **Ready:** The process is loaded into main memory and waiting to be allocated CPU time.
3. **Running:** Instructions of the process are being executed by the CPU.
4. **Waiting (Blocked):** The process is suspended, waiting for an event (e.g., I/O completion or signal).
5. **Terminated:** The process has finished execution and OS resources are being reclaimed.

> ### Question 3
>
> ### Define Schedulers

Schedulers are specialized OS kernel modules that select processes from queues for memory admission and CPU execution:

- **Long-Term Scheduler (Job Scheduler):** Selects batch jobs from disk pool and loads them into memory; controls the *degree of multiprogramming*.
- **Short-Term Scheduler (CPU Scheduler):** Selects a process from the ready queue and assigns the CPU; runs very frequently (milliseconds).
- **Medium-Term Scheduler:** Performs **swapping** (swaps processes out of memory to disk to relieve RAM pressure and swaps them back in later).

> ### Question 4
>
> ### What requirements must be satisfied for a solution to the Critical Section Problem?

1. **Mutual Exclusion:** If process P_i is executing in its critical section, no other process P_j may execute in its critical section simultaneously.
2. **Progress:** If no process is in its critical section and some processes wish to enter, the selection of who enters next cannot be postponed indefinitely.
3. **Bounded Waiting:** There must be a strict limit on how many times other processes can enter the critical section after a process has requested entry, preventing starvation.

> ### Question 5
>
> ### What are the types of Scheduler?

Three Primary Schedulers

Scheduler Type	Primary Function	Execution Frequency
Long-Term (Job)	Loads jobs from disk to RAM; sets multiprogramming degree	Infrequently (seconds/minutes)
Short-Term (CPU)	Allocates CPU core to ready process	Very frequently (milliseconds)
Medium-Term	Swapping processes between RAM and disk	Intermediate / As needed

> ### Question 6
>
> ### Define Thread

Definition & Structure

A **Thread** is the basic unit of CPU utilization, commonly called a **Lightweight Process (LWP)**. It comprises:

- Its own unique **Thread ID (TID)**, **Program Counter (PC)**, **Register set**, and **Stack**.
- It **shares** the Code section, Data section (global variables), and OS resources (open files, signals) with other peer threads belonging to the same parent process.

> **Question 7**
>
> **What does PCB contain?**

Contents of the Process Control Block (PCB)

- **Process State:** Current state (New, Ready, Running, Waiting, Terminated).
- **Process ID (PID):** Unique numerical identifier assigned to the process.
- **Program Counter (PC):** Memory address of the next machine instruction to be executed.
- **CPU Registers:** Accumulators, index registers, stack pointers saved during context switches.
- **CPU Scheduling Information:** Process priority, pointers to scheduling queues.
- **Memory-Management Information:** Base/limit registers, page tables, or segment tables.
- **Accounting Information:** CPU time used, time limits, process account numbers.
- **I/O Status Information:** List of allocated I/O devices and open file descriptors.

> **Question 8**
>
> **What is a Multiprocessor system?**

Definition & Types

A **Multiprocessor System (Tightly-Coupled System)** contains two or more CPU cores that share physical memory, system bus, clock, and peripheral devices to execute processes concurrently.

- **Symmetric Multiprocessing (SMP):** Every processor runs an identical copy of the OS and can schedule any ready process independently (most modern systems).
- **Asymmetric Multiprocessing (Master-Slave):** One master CPU runs the OS and delegates tasks to slave processors.
- **Advantages:** Increased throughput, economy of scale, and increased fault tolerance.

> **Question 9**
>
> **Define Context Switching**

Definition & Properties

Context Switching is the procedure of saving the current state (context) of a running process in its PCB and loading the saved context of the next scheduled process from its PCB, so the CPU can switch execution seamlessly.

- **Pure Overhead:** The system does zero useful computing work during a context switch; execution speed depends strictly on memory speed and register architecture.

> **Question 10**
>
> **List the Thread Libraries**

1. **POSIX Pthreads:** Standard IEEE 1003.1c thread creation and synchronization API implemented in UNIX, Linux, and macOS.
2. **Win32 Threads:** C/C++ thread management library built into Microsoft Windows operating systems.
3. **Java Threads:** Thread API managed directly by the Java Virtual Machine (JVM) using the `Thread` class and `Runnable` interface.

SECTION B — 10 MARKS QUESTIONS

> **Essay Question 1**
>
> **(a) Define Process. Explain Process State Diagram. (b) Explain about Process Schedulers.**

(a) Process & Process State Transition Diagram

A **Process** is a program in execution containing code, stack, data, heap, and CPU registers.

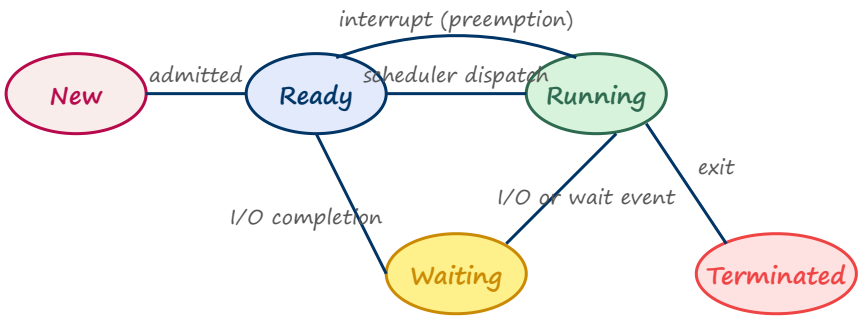

Fig 2.1: Process State Transition Diagram

State Transitions:

- **New → Ready:** Admitted to main memory.
- **Ready → Running:** CPU Scheduler assigns CPU core to the process.
- **Running → Waiting:** Process requests I/O or waits for a synchronization event.
- **Waiting → Ready:** I/O event finishes; process rejoins the ready queue.
- **Running → Ready:** Preempted due to timer interrupt (time slice expired).
- **Running → Terminated:** Program completes execution and calls `exit()`.

(b) Process Schedulers & Context Switching

- **Long-Term Scheduler:** Brings jobs from disk to RAM; balances I/O-bound and CPU-bound processes.
- **Short-Term Scheduler:** Dispatches CPU to ready processes every few milliseconds.
- **Medium-Term Scheduler:** Swaps processes in/out of RAM to prevent page thrashing.
- **Context Switching:** Saves PCB of outgoing process, restores PCB of incoming process.

> **Essay Question 2 (NUMERICAL PROBLEM)**
>
> **Consider 3 processes P1, P2 and P3 requiring 5, 7, 4 time units arriving at 0, 1, 3. Draw Gantt chart and find average waiting time for: (i) Round Robin (quantum=2) (ii) FCFS.**

Given Process Information Table

Process	Burst Time (BT)	Arrival Time (AT)
P1	5	0
P2	7	1
P3	4	3

(i) Round Robin Scheduling (Time Quantum $q = 2$)

Gantt Chart Execution Sequence:

| P1 (0-2) | P2 (2-4) | P1 (4-6) | P3 (6-8) | P2 (8-10) | P3 (10-12) | P1 (12-13) | P2 (13-16) |
0 2 4 6 8 10 12 13 16

Process	Burst	Arrival	Completion Time (CT)	Turnaround Time (TAT = CT - AT)	Waiting Time (WT = TAT - BT)
P1	5	0	13	13 - 0 = 13	13 - 5 = 8
P2	7	1	16	16 - 1 = 15	15 - 7 = 8
P3	4	3	12	12 - 3 = 9	9 - 4 = 5

$$\text{Average Waiting Time (RR, q=2)} = \frac{8 + 8 + 5}{3} = \frac{21}{3} = \mathbf{7.00 \text{ time units}}$$

(ii) First-Come, First-Served (FCFS) Scheduling

Gantt Chart Execution Sequence:

```
|     P1 (0 - 5)     |     P2 (5 - 12)     |     P3 (12 - 16)     |
0                     5                     12                    16
```

Process	Burst	Arrival	Start Time	Completion Time (CT)	Turnaround (TAT = CT - AT)	Waiting (WT = TAT - BT)
P1	5	0	0	5	5 - 0 = 5	5 - 5 = **0**
P2	7	1	5	12	12 - 1 = 11	11 - 7 = **4**
P3	4	3	12	16	16 - 3 = 13	13 - 4 = **9**

$$\text{Average Waiting Time (FCFS)} = \frac{0 + 4 + 9}{3} = \frac{13}{3} \approx \mathbf{4.33 \text{ time units}}$$

Essay Question 3

Explain CPU Scheduling Algorithms with examples

1. **FCFS (First-Come, First-Served):** Non-preemptive. FIFO queue. Simple but suffers from the **Convoy Effect** (short processes waiting behind a long burst process).
2. **SJF (Shortest Job First):** Picks the process with the smallest burst time. Mathematically guarantees the **optimal (minimum) average waiting time**. Can be preemptive (SRTF - Shortest Remaining Time First) or non-preemptive.
3. **Priority Scheduling:** Allocates CPU to the highest priority process. Can cause **starvation** (indefinite blocking) of low-priority processes; solved by **Aging** (gradually increasing priority over time).
4. **Round Robin (RR):** Preemptive; designed for time-sharing systems. Each process gets a fixed time quantum (q); ideal response time.
5. **Multilevel Queue Scheduling:** Partitions the ready queue into distinct foreground (interactive RR) and background (batch FCFS) queues with fixed priorities.
6. **Multilevel Feedback Queue:** Allows processes to migrate between queues based on CPU-burst behavior (I/O-heavy processes move to high priority; CPU-heavy processes sink to lower queues).

> **Essay Question 4 (NUMERICAL PROBLEM)**
>
> **(a) Explain about Scheduling Criteria. (b) Evaluate FCFS CPU Scheduling for P1(24), P2(3), P3(5), P4(6) all arriving at t=0.**

(a) CPU Scheduling Criteria

- **CPU Utilization:** Percentage of time CPU is busy (aim: 40%–90%).
- **Throughput:** Number of processes completed per unit time (maximize).
- **Turnaround Time (TAT):** Submission to completion: $TAT = CT - AT$ (minimize).
- **Waiting Time (WT):** Total time in ready queue: $WT = TAT - BT$ (minimize).
- **Response Time (RT):** Time from arrival to first CPU response (minimize).

(b) Numerical Evaluation of FCFS

Gantt Chart:

```
|        P1 (0 - 24)        | P2 (24-27) | P3 (27-32) | P4 (32-38) |
0                          24           27           32           38
```

Process	Burst	Start Time	Completion Time (CT)	Turnaround (TAT)	Waiting Time (WT)
P1	24	0	24	24	0
P2	3	24	27	27	24
P3	5	27	32	32	27
P4	6	32	38	38	32

$$\text{Average Waiting Time} = \frac{0 + 24 + 27 + 32}{4} = \frac{83}{4} = \mathbf{20.75 \text{ time units}}$$

Observation: Demonstrates the **Convoy Effect** — short processes P2, P3, P4 endure long waiting times behind P1.

> **Essay Question 5 (NUMERICAL PROBLEM)**
>
> **Evaluate SJF CPU Scheduling for P1(8,0), P2(4,1), P3(9,2), P4(5,3)**

Step-by-Step Non-Preemptive SJF Execution

- At $t = 0$: Only P1 is available → Run P1 from $t = 0$ to $t = 8$.
- At $t = 8$: P2 ($BT=4$), P3 ($BT=9$), P4 ($BT=5$) have arrived. Shortest is P2 → Run P2 from $t = 8$ to $t = 12$.
- At $t = 12$: Remaining are P4 ($BT=5$) and P3 ($BT=9$). Shortest is P4 → Run P4 from $t = 12$ to $t = 17$.

- At $t = 17$: Run P3 from $t = 17$ to $t = 26$.

Gantt Chart:

```
|     P1 (0 - 8)     |     P2 (8 - 12)     |     P4 (12 - 17)     |     P3 (17 - 26)     |
0                     8                     12                     17                     26
```

Process	Burst	Arrival	Completion Time (CT)	Turnaround (TAT = CT - AT)	Waiting Time (WT = TAT - BT)
P1	8	0	8	8 - 0 = 8	8 - 8 = **0**
P2	4	1	12	12 - 1 = 11	11 - 4 = **7**
P4	5	3	17	17 - 3 = 14	14 - 5 = **9**
P3	9	2	26	26 - 2 = 24	24 - 9 = **15**

$$\text{Average Waiting Time (SJF)} = \frac{0 + 7 + 9 + 15}{4} = \frac{31}{4} = \mathbf{7.75 \text{ time units}}$$

> **Essay Question 6 (NUMERICAL PROBLEM)**
>
> **Evaluate Round Robin CPU Scheduling (Time Slice = 3 ms) for P1(10,5), P2(5,3), P3(18,0), P4(6,4)**

Step-by-Step RR Simulation ($q = 3$)

Ready Queue Tracking:

- $t = 0$: Ready queue = [P3]. Run P3 (0-3). Remaining: P3(15).
- $t = 3$: P2 arrives ($AT=3$). Queue = [P2, P3]. Run P2 (3-6). Remaining: P2(2). During this time, P4 arrives at $t=4$ and P1 arrives at $t=5$.
- $t = 6$: Queue = [P3, P4, P1, P2]. Run P3 (6-9). Remaining: P3(12).
- $t = 9$: Queue = [P4, P1, P2, P3]. Run P4 (9-12). Remaining: P4(3).
- $t = 12$: Queue = [P1, P2, P3, P4]. Run P1 (12-15). Remaining: P1(7).
- $t = 15$: Queue = [P2, P3, P4, P1]. Run P2 (15-17). **P2 Completes at $t=17$!**
- $t = 17$: Queue = [P3, P4, P1]. Run P3 (17-20). Remaining: P3(9).
- $t = 20$: Queue = [P4, P1, P3]. Run P4 (20-23). **P4 Completes at $t=23$!**
- $t = 23$: Queue = [P1, P3]. Run P1 (23-26). Remaining: P1(4).
- $t = 26$: Queue = [P3, P1]. Run P3 (26-29). Remaining: P3(6).
- $t = 29$: Queue = [P1, P3]. Run P1 (29-32). Remaining: P1(1).
- $t = 32$: Queue = [P3, P1]. Run P3 (32-35). Remaining: P3(3).
- $t = 35$: Queue = [P1, P3]. Run P1 (35-36). **P1 Completes at $t=36$!**

- $t = 36$: Queue = [P3]. Run P3 (36-39). **P3 Completes at $t=39$!**

Process	Burst	Arrival	Completion Time (CT)	Turnaround Time (TAT = CT - AT)	Waiting Time (WT = TAT - BT)
P2	5	3	17	17 - 3 = 14	14 - 5 = 9
P4	6	4	23	23 - 4 = 19	19 - 6 = **13**
P1	10	5	36	36 - 5 = 31	31 - 10 = **21**
P3	18	0	39	39 - 0 = 39	39 - 18 = **21**

$$\text{Average Waiting Time (RR, q=3)} = \frac{9 + 13 + 21 + 21}{4} = \frac{64}{4} = \mathbf{16.00 \text{ ms}}$$

> **Essay Question 7**
>
> **Explain in detail Inter-Process Communication (IPC)**

1. Introduction to IPC

Inter-Process Communication (IPC) is a mechanism provided by the OS that allows cooperating processes to exchange data and synchronization signals.

2. Two Fundamental IPC Models

(a) Shared Memory Model

- A region of physical memory is mapped into the address spaces of multiple processes.
- Processes read and write directly to this shared buffer.
- **Advantage:** Maximum speed (no kernel intervention after setup).
- **Disadvantage:** Synchronization must be managed explicitly by the programmer (e.g., using semaphores) to prevent race conditions.

(b) Message Passing Model

- Processes exchange messages using kernel system calls: `send(message)` and `receive(message)`.
- **Advantage:** Ideal for distributed systems; synchronizations are handled automatically by the OS.
- **Disadvantage:** Slower due to kernel context-switch and copying overhead.

3. Comparison Table

Basis	Shared Memory	Message Passing
Speed	Fast (memory access speed)	Slower (kernel overhead per message)

Synchronization	Handled by programmer	Handled automatically by OS
Application	Large data exchange on single machine	Distributed systems / small messages

> **Essay Question 8**
>
> **Discuss about Threading Issues**

1. **The `fork()` and `exec()` Semantics:** If one thread calls `fork()`, does the OS duplicate all threads or only the calling thread? (UNIX provides two versions of `fork()`).
2. **Signal Handling:** Signals (e.g., `SIGSEGV`, `SIGINT`) can be delivered to: (a) the thread that caused it, (b) every thread, or (c) a dedicated signal-handling thread.
3. **Thread Cancellation:**
   - *Asynchronous:* Terminates target thread immediately (risk of leaving shared resources in inconsistent states).
   - *Deferred:* Target thread checks periodically at safe **cancellation points**.
4. **Thread-Local Storage (TLS):** Enables each thread to have its own unique instance of global data.
5. **Scheduler Activations:** Communication bridge between user thread library and kernel via Lightweight Processes (LWPs).

> **Essay Question 9**
>
> **Write about Operations on Processes**

1. Process Creation

- Parent processes create child processes using `fork()`, forming a tree of processes (rooted at `systemd` or `init`).
- **Resource Sharing:** Parent and child can share all, some, or zero resources.
- **Execution Options:** Parent executes concurrently with child, or parent calls `wait()` until child terminates.

2. Process Termination

- Process finishes its last statement and calls `exit()`.
- **Orphan Process:** A process whose parent has terminated while the child is still running (adopted by `init`).
- **Zombie Process:** A process that has terminated, but whose parent has not yet called `wait()` (remains in process table).

> ### Essay Question 10
>
> ### What is a Thread? Discuss about Multithreading Models

1. Definition & Benefits

A **Thread** is a basic CPU execution unit sharing memory and resources with peer threads.

- **Benefits:** Responsiveness, Resource Sharing, Economy (cheaper than processes), Scalability across multi-core processors.

2. Multithreading Models

1. **Many-to-One Model:** Maps many user-level threads to a single kernel thread. If one user thread makes a blocking call, the entire process blocks. Cannot exploit multi-core CPUs. (e.g., Green Threads).
2. **One-to-One Model:** Maps each user thread to its own dedicated kernel thread. Provides true parallelism on multiprocessors; creation overhead is higher. (e.g., Windows, Linux).
3. **Many-to-Many Model:** Multiplexes many user threads onto an equal or smaller number of kernel threads, combining flexibility and high throughput.

LAST-MINUTE EXAM REVISION (UNIT 2)

CPU Scheduling Formula Reference

Metric / Algorithm	Mathematical Formula / Expression
Turnaround Time (TAT)	$TAT = \text{Completion Time (CT)} - \text{Arrival Time (AT)}$
Waiting Time (WT)	$WT = \text{Turnaround Time (TAT)} - \text{Burst Time (BT)}$
Response Time (RT)	$RT = \text{First CPU Allocation Time} - \text{Arrival Time (AT)}$
Average Waiting Time	$\frac{\sum WT}{\text{Total Number of Processes}}$
SJF Advantage	Guarantees minimum average waiting time for any given set of processes